拼音输入法 实验报告

张家盈
2023011382
生命科学学院

i. 实验环境

编程语言
python
version -- Python 3.13.2

IDE
PyCharm

运行测试
PyCharm
本地CMD

操作系统
Windows 11

库
均使用 Python 3.13.2 标准库，无需安装第三方库：

- os
- sys
- json
- math
- re
- ast
- collections.defaultdict

ii. 语料库和数据预处理

使用新浪新闻语料库（ `sina_news_gbk` ）作为训练数据来源，数据格式为包含新闻内容的 JSON 列表或类 JSON 格式的文本文件（我在处理的时候注意到我的ide会对SMP2020的格式报错，因为它使用单引号括键值而不是双引号，因此采用了别的库读入为字典）。预处理过程包括以下步骤：

1. 过滤与清洗

首先从文本中提取所有**连续的中文字符片段**（使用正则表达式匹配 Unicode 范围 `[\u4e00-\u9fff]` ），然后**筛选出只包含国家标准一、二级汉字的子串。**
字符合法性由提供的《一二级汉字表.txt》决定。

对于bigram的训练，每个被保留的子串长度需大于 1；对于trigram的训练，子串需大于 2。

2. 多音字处理

根据预先构建好的汉字-拼音映射表（ `char_to_py.txt` ）来处理多音字。在统计语言模型时：

- 对于任意字符，会考虑它可能对应的所有拼音；
- 对于二元组、三元组，会考虑所有的拼音组合，如对于二元组，一组二连字符 (c_1, c_2) 会对应 $|Q_1| \cdot |Q_2|$ 种拼音组合（其中 Q_i 是第 i 个字符对应的拼音列表）；
- 为保证整体频率总和一致，采用**均分策略**，即每种拼音组合得到 $\frac{1}{|P_1| \cdot |P_2|}$ 的贡献；
- 三元组，(c_1, c_2, c_3) 对应 $|Q_1| \cdot |Q_2| \cdot |Q_3|$ 种组合，均分策略同二元组。

3. 数据结构格式

对一元、二元，将统计结果转换为匹配评测系统OJ的 JSON 格式（为了适应提交OJ的二元程序）：

```
{
  "ni": {
    "words": ["你", "呢", "泥"],
    "counts": [1234, 56, 12]
  }
}
```

对于三元训练结果，保存为如下格式的 JSON 文件（`3_word.txt`）：

```
{
  "ni hao ma": {
    "你 好 吗": 3.5,
    "妮 好 嘛": 1.0
  },
  ...
}
```

由于三元模型的组合总量较大，生成的原始字典体积极大，因此最终训练时**仅保留出现次数超过 1 的组合**，以压缩模型体积、避免不必要的噪声。开发者尝试过保留所有出现次数1的组合，会造成训练结果文件存储时截断，因太大无法存储。

4. 输入输出格式与日志输出

为了兼容作业评测，程序严格遵循 `stdin` → `stdout` 的输入输出形式，所有训练过程相关信息通过 `sys.stderr` 输出，防止干扰系统输出评测。

iii. 二元模型的拼音输入法

a. 基本思路、公式推导和算法原理

基于字的Bigram模型实现，输入拼音序列，输出对应最可能的汉字序列。

最终目标是最大化整个句子的概率：

$$\text{Viterbi}(i, w_i) = \max_{w_{i-1}} \left[\text{Viterbi}(i-1, w_{i-1}) \cdot P(w_i \mid w_{i-1}) \cdot P(w_i \mid p_i)\right]$$

- $P(w_i \mid p_i)$ 表示给定拼音 p_i，对应汉字 w_i 的先验概率；
- $P(w_i \mid w_{i-1})$ 是语言模型中的转移概率。

字与字之间的转移概率 $P(w_i \mid w_{i-1})$ 使用条件频率估计方法计算：

$$P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

其中：

- $\text{Count}(w_{i-1}, w_i)$ 表示 w_{i-1} 和 w_i 连续出现的次数（bigram 频率）；
- $\text{Count}(w_{i-1})$ 表示 w_{i-1} 出现的总次数（unigram 频率）。

为了方便处理概率的“乘积”，本项目对概率项取对数处理。在实际实现中，采用了线性插值的方式将语言模型概率 $P(w_i \mid w_{i-1})$ 和拼音模型概率 $P(w_i \mid p_i)$ 进行融合。具体做法如下：

先计算两者的原始概率：

- $P(w_i \mid w_{i-1})$ 来自 bigram 模型统计；
- $P(w_i \mid p_i)$ 来自输入拼音对应的候选字频率；

然后使用线性插值：

$$P(w_i) = \lambda \cdot P(w_i \mid w_{i-1}) + (1-\lambda) \cdot P(w_i \mid p_i)$$

最后再对结果取对数：

$$\log P(w_i) = \log\left(\lambda \cdot P(w_i \mid w_{i-1}) + (1-\lambda) \cdot P(w_i \mid p_i)\right)$$

这一操作避免了在 log 域中直接插值导致的数值不一致问题，确保了概率计算的数学合理性。

b. 实验效果

Bigram模型

- 总训练时间（bigram）: 2027.74s
- 生成句子时间（501条）: 10.97s
- 总句数: 501
- 句准确率: 40.72%
- 总字符数: 5235
- 字准确率: 84.20%
- 平均每句字符正确率: 82.89%

c. 实例分析

错误案例分析

1. 拼音输入： `wei ji bai ke shi yi ge wang luo bai ke quan shu xiang mu`
   - 预测输出： 为几百科室一个网络百科拳术项目
   - 标准答案： 维基百科是一个网络百科全书项目
   - **问题分析**：将“维基百科”误识为“为几百科”；bigram 模型缺乏跨“双字词”的意识，对“百科全书”这种常见四字词组也处理成了“百科拳术”，“百科”和“拳术”都是常见的双字词。
2. 拼音输入： `yong yuan ju bu zui you jie de ren dao bu liao quan ju zui you`
   - 预测输出： 用原局部最有家的人到不了全局最有
   - 标准答案： 永远局部最优解的人到不了全局最优
   - **问题分析**：这里注意到一个有趣的，“最有家”的“家”读音不是或者至少不经常是 `jie` ,因为本项目的多音字计数并没有实现“常用音”和不常用音的区分而是平均计数，这是可能出现的情况。
3. 拼音输入： `zhong guo chen wen ying dui mao yi mo ca`
   - 预测输出： 中国陈文应对贸易摩擦
   - 标准答案： 中国沉稳应对贸易摩擦
   - **问题分析**：误将“沉稳”识别为“陈文”，出现误判为人名类错误；bigram 无法结合“应对”形成合适搭配，缺乏句子级全局判断。
4. 拼音输入： `xi wang wo de ai dou zhuan ji xiao liang di yi`
   - 预测输出： 希望我的爱读专机销量的一
   - 标准答案： 希望我的爱豆专辑销量第一
   - **问题分析**：“爱读专机”完全语义混乱，说明 bigram 对于“爱豆”、“专辑”这些现代词较弱，可能跟训练语料多是新闻稿相关。
5. 拼音输入： `ni de li jie shi dui de`
   - 输出： 你的理解释对的
   - 标准： 你的理解是对的
   - **问题分析**： “理解释”是错误的三元组合，但是“理解”和“解释”都是正确的二元组合，bigram 没有能力辨别三元组的正确性。

正确案例分析：

1. 拼音输入： `zhe xue shi yi qie xue ke de ji chu`
   - 预测输出： 哲学是一切学科的基础
   - 标准答案： 哲学是一切学科的基础
   - **分析**：简单句结构，词汇常见，bigram 可顺利匹配。
2. 拼音输入： `qing shan lv shui jiu shi jin shan yin shan`
   - bigram 输出： 青山绿水就是金山银山
   - 标准答案： 青山绿水就是金山银山
   - **分析**：这句是国家级高频宣传口号，bigram 在训练语料（偏新闻/政务）中出现概率极高，“青山绿水”“金山银山”都是固定搭配，组合顺畅自然。
3. 拼音输入： `ren min qun zhong xi wen le jian`
   - bigram 输出： 人民群众喜闻乐见
   - 标准答案： 人民群众喜闻乐见
   - **分析**：也是典型官媒句式，bigram 训练中高度频繁地学到“人民群众”“喜闻乐见”这类四字搭配，且是短句，容易得到正确输出。

原因分析

Bigram 模型由于仅考虑前一个字的上下文信息，因此在较长、结构复杂、歧义性强，尤其需要“超过双字词分析”的句子中更容易出现偏差。
这导致句准确率偏低，尽管字符层面有较高的匹配率，但整体结构往往被打乱。

iv. 思考题

a. 语料编码：GBK 与 UTF-8 的区别

- **GBK** 是中国特有的字符编码方式，基于 GB2312，采用1～2 字节变长编码。它支持所有简体和部分繁体汉字，适用于中文本地应用，如 Windows 系统、新闻网站数据库等。
- **UTF-8** 是一种国际通用的 Unicode 编码方式，兼容 ASCII，采用 1～4 字节变长编码，能够表示世界上几乎所有文字。常见于网页、移动端、跨平台开发中，Python 默认使用 UTF-8。

举例及应用场景：

- 新浪新闻语料文件使用 GBK 编码，需指定编码方式才能正确解析（encoding='gbk'）。
- GitHub 上的项目、网络 API、前端 HTML 页面通常使用 UTF-8 编码。

其他常见编码：

- **UTF-16**：是Unicode最早的编码方式，采用固定或变长双字节表示字符，适用于大型文档处理系统，如 Windows 内部或 Word。
- **ISO-8859-1**：单字节编码，覆盖了西欧主要语言的字符，主要用于早期网页或英文文档。

b. 算法设计：Viterbi 的时间与空间复杂度分析

设拼音输入序列长度为 n，每个拼音对应不超过 V 个候选汉字，所有拼音候选列表均提前缓存。

Bigram 模型：

- 状态定义：第 i 个位置保留所有可能的当前字符 w_i。
- 对于每个候选字 w_i，需要遍历前一个位置所有候选字 w_{i-1}，并使用插值平滑估计：

$$\log P(w_i \mid w_{i-1}, p_i) = \log\left(\lambda P(w_i \mid w_{i-1}) + (1-\lambda) P(w_i \mid p_i)\right)$$

- 每步转移计算当前 V 个状态和前一个字的 V 个状态，时间复杂度为 $O(V^2)$。

总时间复杂度：

$$O(n \cdot V^2)$$

空间复杂度：

- 实现中为每一位置都保留了完整的 DP 值数组（`dp[i][ci]`）和回溯路径数组（`back[i][ci]`），未使用滚动数组优化。
- 整体空间消耗为：

$$O(n \cdot V)$$

Trigram 模型：

- 状态定义：每一位置保留所有可能的二元组状态 (w_{i-2}, w_{i-1})，每个状态对应一个当前字符 w_i。
- 为计算 $P(w_i \mid w_{i-2}, w_{i-1})$，系统枚举所有 V^2 个历史状态和当前 V 个候选字，插值如下：

$$P(w_i \mid \cdot) = \alpha P(w_i \mid w_{i-2}, w_{i-1}) + \lambda P(w_i \mid w_{i-1}) + (1-\alpha-\lambda) P(w_i \mid p_i)$$

- 每步最多需要 $V^2 \times V$ 次计算。

总时间复杂度：

$$O(n \cdot V^3)$$

空间复杂度：

- 实现中未使用状态压缩，保留了每一时刻所有 $O(V^2)$ 的二元状态转移路径。
- DP 表为长度 n，每步保存 V^2 个状态，回溯路径同样。

$$O(n \cdot V^2)$$

c. 输入输出：格式规范

在 OJ 系统中评测时，程序的标准输出（stdout）必须严格只包含答案，不能混入任何调试信息或输入提示。因此，输入读取方式必须避免输出与输入内容交叉的问题。

我们对比两种读取方式的行为：

代码 A：边读边输出

```
while not EOF:
    a = input()
    print(a)
```

终端行为：

```
>>> 1
1
>>> 2
2
```

由于 `input()` 逐行读取并立刻打印，输入和输出**交错出现**，评测系统在抓 stdout 时可能误将输入也算进去，从而输出不匹配。

代码 B：先全部读取再统一输出

```
lst = []
while not EOF:
    a = input()
    lst.append(a)
for x in lst:
    print(x)
```

终端行为：

```
>>> 1
>>> 2
1
2
```

尽管终端仍会回显输入，但程序的 stdout 中只包含最终的输出内容，不会交叉掺杂输入与输出，更适合用于自动评测。

本项目实现 & 反思

本项目采用的输入处理方式为：

```
import sys
for line in sys.stdin:
    ...
    print(...)
```

- 不会在 stdout 中掺杂输入内容**，因为输入不是交互式；
- 输出内容始终整洁，不会出现 `输入-输出-输入-输出` 的交叉模式；
- 可与 `< input.txt > output.txt` 配合使用；
- 所有调试信息（如训练进度、计时）均使用 `sys.stderr` 输出，不干扰评测。

选做部分

v. 其他模型：三元语言模型

a. 算法思路

公式推导

在三元语言模型中，本项目并未使用中文分词后的“词语”为基本单元，而是仍然以单字为单位构建语言模型。即三元组表示的是连续三个汉字字符。这种模型能够捕捉字符级的上下文信息。
本项目综合使用 trigram、bigram 与拼音条件概率，通过线性插值的方法对每个候选字进行打分。递推公式如下：

$$\mathrm{Viterbi}(i, w_{i-1}, w_i) = \max_{w_{i-2}} \left[\mathrm{Viterbi}(i-1, w_{i-2}, w_{i-1}) + \log P(w_i \mid w_{i-2}, w_{i-1}, p_i)\right]$$

其中：

$$P(w_i \mid w_{i-2}, w_{i-1}, p_i) = \alpha \cdot P(w_i \mid w_{i-2}, w_{i-1}) + \lambda \cdot P(w_i \mid w_{i-1}) + (1 - \alpha - \lambda) \cdot P(w_i \mid p_i)$$

从三个来源获取概率：

- $P(w_i \mid w_{i-2}, w_{i-1})$：由 trigram 出现频率统计得到；
- $P(w_i \mid w_{i-1})$：由 bigram 统计得到；
- $P(w_i \mid p_i)$：由拼音到汉字的先验概率统计得到。

整个插值操作在概率域中进行，得到最终的综合概率后，再取对数传入 Viterbi 动态规划中用于累加。

回溯策略说明

在三元组 Viterbi 解码过程中，路径信息依赖于前两个字构成的状态 (w_{i-2}, w_{i-1})，因此动态规划的状态表示为当前字和其前两个字的组合。每个位置记录的是从上一步状态转移而来的最优分数及来源路径。

解码完成后，从最后一步的所有状态中选取分数最高的状态作为终点，随后通过回溯路径依次找出其来源状态，但每次回溯只退一步，相当于从w_i退回二元组的更靠近它的那个w_{i-1}，直到抵达起点。

b. 实验效果

Trigram模型

- 总训练时间（trigram）: 2549.46s
- 生成句子时间（501条）: 291.26s
- 总句数: 501
- 句准确率: 65.07%
- 总字符数: 5235
- 字准确率: 91.73%
- 平均每句字符正确率: 90.91%

总结分析：
尽管 trigram 训练和解码时间明显增加（训练时长在“bigram已经训练好的文件基础上”还多出 500s，解码时间接近 bigram 的 27 倍），但在准确率上表现显著更优：句准确率提升约 **24%**，字符准确率提升近 **8%**，每句平均字符准确率提升 **超过8%**，说明 trigram 更适合处理结构复杂、词组较长的语料，尤其在本项目以新闻稿训练为主的场景中效果明显。

c. 案例分析

trigram 修正成功案例

1. 拼音： `wei ji bai ke shi yi ge wang luo bai ke quan shu xiang mu`

- bigram 输出： 为几百科室一个网络百科拳术项目
- trigram 输出： 维基百科是一个网络百科全书项目
- 标准答案： 维基百科是一个网络百科全书项目

分析：
trigram 成功捕捉了“维基百科”和“百科全书”两个命名词块的搭配，避免了 bigram 只分析“双字词”的错误拼接（如“百科拳术”）。这是 n-gram 提升上下文范围后的典型成功案例。

2. 拼音： `zhong guo chen wen ying dui mao yi mo ca`

- bigram 输出： 中国陈文应对贸易摩擦
- trigram 输出： 中国沉稳应对贸易摩擦
- 标准答案： 中国沉稳应对贸易摩擦

分析：
trigram 成功判断“沉稳应对”为一组合理搭配，而没有误选“陈文”这种人名（人名搭配“应对”语义不合）。这说明 trigram 在语义自然流畅度上更强。这句话的字词组合在时政新闻（即训练语料的多数）中也常见。

3. 拼音： `yi qing mian qian zhong sai liang guo zai ci xie shou gong ke shi jian`
   - bigram 输出： 疫情面前中塞两国再次携手工课时间
   - trigram 输出： 疫情面前中塞两国再次携手共克时艰
   - 标准答案： 疫情面前中塞两国再次携手共克时艰

分析：
尽管两种模型都基于同样的新闻语料训练，bigram 受限于仅能看一个前词，错误地把“共克”切成了“工课”，又把“时艰”拆成“时间”，在字面相似度上被误导；而 trigram 能看两个前词，识别出“携手—共克—时艰”是疫情报道中高频、三字以上的固定搭配，因此完美恢复了原句。这体现了 trigram 在处理长词组搭配、尤其是新闻中常用套话时的结构优势。

trigram 仍然错误案例

1. 拼音： `yong yuan ju bu zui you jie de ren dao bu liao quan ju zui you`

- bigram 输出： 用原局部最有家的人到不了全局最有
- trigram 输出： 用原局部最有价的人道补了全剧最有
- 标准答案： 永远局部最优解的人到不了全局最优

分析：
trigram 在这里单句的字准确率甚至有所下降，我们可以看到这里出现了一点高频的三字组合，如“最有价”。本句的表述并非“官方措辞”，模型在训练中可能对其不够熟悉。

2. 拼音： `xi wang wo de ai dou zhuan ji xiao liang di yi`

- bigram 输出： 希望我的爱读专机销量的一
- trigram 输出： 希望我的爱读专辑销量第一
- 标准答案： 希望我的爱豆专辑销量第一

分析：
trigram 修正了“销量的一”变回“销量第一”，但“爱豆”还是错写成了“爱读”。原因可能是，“爱豆”是近几年才流行的音译词，如果语料较旧，模型可能不认识这个词；“爱读”更常出现在传统语料中，频次高，自然被优先选中。

可见 trigram **在固定搭配、命名实体、句法流畅度上表现更好**，但**面对拼音歧义、语义模糊或冷门网络词汇**时，依然会失败。

vii. 额外评估指标分析：

平均每句字符正确率（Average per-sentence char accuracy）

定义与意义：
该指标表示对每一句测试句分别计算其字符正确率（即每句中被预测正确的字符数 / 该句总字符数），再对所有句子求平均。
不同于整体字符正确率（统计所有字符的准确率），这个指标能够更公平地反映模型对**每个句子处理质量的一致性**，避免“长句子主导”导致的评价偏差。

例如：

- 如果一个模型在大部分短句中表现糟糕，但在长句中几乎全对，那么整体字符准确率可能仍较高；
- 而“平均每句字符正确率”会更敏感地捕捉到这种偏差，因为每个句子等权计算。

实验结果比较：

模型类型	字符准确率	平均每句字符正确率
Bigram	84.20%	82.89%
Trigram	91.73%	90.91%

分析：

- Trigram 模型在两个指标上都显著优于 Bigram，提升幅度在约 **7.5% 左右**。
- 两种指标的差值（整体字符准确率 - 平均每句字符正确率）在两种模型中分别为：
  - **Bigram：** 84.20% - 82.89% = 1.31%
  - **Trigram：** 91.73% - 90.91% = 0.82%

结论：
Trigram 不仅整体字符识别能力更强，也在不同长度句子间表现更稳定，平均句表现更均衡，说明其上下文建模能力更优，适合新闻类语料中结构复杂的文本预测任务。

vii. 实验反馈

感受大致是工作量比想象的大很多很多。
感觉算法思路很简单，代码比想象的难写很多，主要debug好耗时间，oj也总是过不了，时间超限完空间超，加了缓存就是运行错误，本地运行结果很好一上oj一崩再崩。
终于过了oj，训练数据也没那么容易，最开始同样的代码同学训练出的数据集能跑的很好，自己的数据集就总是不行，最后发现不能把语料全堆在一起，需要按照不是中文字的符号分开“中文语段”。然后就是刚开始多音字也没处理好，甚至导致从拼音汉字对照表下来的输入，有的多音字只有一个音，准确率一度掉到百分之零点几。
三元组的拓展也没有想象中那么简单，到底是动态规划比较容易数组越界吧，debug弄了好久。这次实验写到最后实在精疲力竭，实在倒腾不动调参了，问了问同学用着效果不错的平滑概率的参数，效果看起来不错，这个项目就先暂且这么用着了。此外在三元组的处理上没有特别考虑时间和空间复杂度，应该是还有一些优化空间的。